Ananya Yallapragada
019142383

Homework 6 Report

Section 0 - Personal Configuration and Domain

Value	Definition
SID4	2383
PORT_BASE	8583
PREFIX	s2383
SEED	2383
VERIFY_SEED	262383
DOMAIN_ID	7
Hardware	MacBook Pro, Apple M4 (10-core CPU), 16 GB RAM
Python version	3.11.9
Tagged commit hash	68e255789a427c4cc2021b682e809e3025e17cc0

Github - https://github.com/Lyra0710/data260-2383/tree/hw6

(Please note that since the commit hash needs to be included in the report, a new commit will be created when the report is added again with the hash included. The tag link here is before the hash was added. The updated tag is pasted in the comments on canvas.).

Ananya Yallapragada
019142383

Part 1 - MongoDB Document API

I. Database

I extended the existing Community Sports League Fixtures application with a MongoDB document representation. The MongoDB database is s2383_doc, based on my assigned prefix s2383, and the Part 1 collection is fixtures.

Mongo db connection - mongo_db.py

```python
import os

from dotenv import load_dotenv
from pymongo import MongoClient

load_dotenv()

MONGO_URI = os.getenv("MONGO_URI")

if not MONGO_URI:
    raise ValueError("MONGO_URI environment variable is not set")

client = MongoClient(MONGO_URI)

db = client["s2383_doc"]
fixtures_collection = db["fixtures"]
```

Schema - Updating schemas.py

```python
class FixtureStatus(str, Enum):
    scheduled = "scheduled"
    completed = "completed"
    cancelled = "cancelled"


class SportType(str, Enum):
    soccer = "soccer"
    basketball = "basketball"
    tennis = "tennis"
    pickleball = "pickleball"


class DocumentFixtureCreate(BaseModel):
    fixture_name: str = Field(min_length=1, max_length=100)
    description: str | None = None
    status: FixtureStatus = FixtureStatus.scheduled
    sport_type: SportType
    scheduled_at: datetime


class DocumentFixtureUpdate(BaseModel):
    fixture_name: str | None = Field(default=None, min_length=1, max_length=100)
    description: str | None = None
    status: FixtureStatus | None = None
    sport_type: SportType | None = None
    scheduled_at: datetime | None = None
```

Ananya Yallapragada
019142383

```
(.venv) [2026-10-15 21:35:18 PDT] annie@mac ~/Library/CloudStorage/OneDrive-Personal/Academics/Masters/SJSU/DATA-260/Homework/data260-2383/code/web_application % python -c "from mongo_db import client; print(client.admin.command('ping'))"
{'ok': 1.0}
```

Fixture validator - init_mongo.py



```
fixture_validator = {
    "$jsonSchema": {
        "bsonType": "object",
        "required": [
            "fixture_name",
            "status",
            "sport_type",
            "scheduled_at",
            "created_at",
            "updated_at",
        ],
        "properties": {
            "fixture_name": {
                "bsonType": "string",
                "minLength": 1,
                "maxLength": 100,
            },
            "description": {
                "bsonType": ["string", "null"],
            },
            "status": {
                "enum": ["scheduled", "completed", "cancelled"],
            },
            "sport_type": {
                "enum": [
                    "soccer",
                    "basketball",
                    "tennis",
                    "pickleball",
                ],
            },
            "scheduled_at": {
                "bsonType": "date",
            },
            "created_at": {
                "bsonType": "date",
            },
            "updated_at": {
                "bsonType": "date",
            },
        },
    }
}
```

```
(.venv) [2026-10-15 21:36:08 PDT] annie@mac ~/Library/CloudStorage/OneDrive-Personal/Academics/Masters/SJSU/DATA-260/Homework/data260-2383/code/web_application % python -c "from mongo_db import db; print(db.command('listCollections', filter={'name': 'fixtures'})['cursor']['firstBatch'][0]['options']['validator'])"
{'$jsonSchema': {'bsonType': 'object', 'required': ['fixture_name', 'status', 'sport_type', 'scheduled_at', 'created_at', 'updated_at'], 'properties': {'fixture_name': {'bsonType': 'string', 'minLength': 1, 'maxLength': 100}, 'description': {'bsonType': ['string', 'null']}, 'status': {'enum': ['scheduled', 'completed', 'cancelled']}, 'sport_type': {'enum': ['soccer', 'basketball', 'tennis', 'pickleball']}, 'scheduled_at': {'bsonType': 'date'}, 'created_at': {'bsonType': 'date'}, 'updated_at': {'bsonType': 'date'}}}}
```

Ananya Yallapragada
019142383

II. API Implementation

document_fixtures.py - All handlers

```python
@router.post("", status_code=201)
def create_fixture(fixture: DocumentFixtureCreate):
    now = datetime.now(timezone.utc)

    document = fixture.model_dump()
    document.update(created_at=now, updated_at=now)

    try:
        result = fixtures_collection.insert_one(document)
    except PyMongoError:
        raise HTTPException(
            status_code=500,
            detail="Failed to create fixture",
        )

    document["_id"] = result.inserted_id

    return fixture_response(document)


@router.get("")
def list_fixtures():
    try:
        fixtures = fixtures_collection.find().sort("scheduled_at", 1)
        return [fixture_response(fixture) for fixture in fixtures]
    except PyMongoError:
        raise HTTPException(
            status_code=500,
            detail="Failed to retrieve fixtures",
        )
```

```python
@router.get("/{fixture_id}")
def get_fixture(fixture_id: str):
    try:
        fixture = fixtures_collection.find_one(
            {"_id": fixture_object_id(fixture_id)}
        )
    except PyMongoError:
        raise HTTPException(
            status_code=500,
            detail="Failed to retrieve fixture",
        )

    if fixture is None:
        raise HTTPException(status_code=404, detail="Fixture not found")

    return fixture_response(fixture)


@router.put("/{fixture_id}")
def update_fixture(fixture_id: str, fixture: DocumentFixtureUpdate):
    updates = fixture.model_dump(exclude_unset=True)

    if not updates:
        raise HTTPException(status_code=400, detail="No update fields provided")

    updates["updated_at"] = datetime.now(timezone.utc)

    try:
        result = fixtures_collection.find_one_and_update(
            {"_id": fixture_object_id(fixture_id)},
            {"$set": updates},
            return_document=ReturnDocument.AFTER,
        )
    except PyMongoError:
        raise HTTPException(
            status_code=500,
            detail="Failed to update fixture",
        )

    if result is None:
        raise HTTPException(status_code=404, detail="Fixture not found")

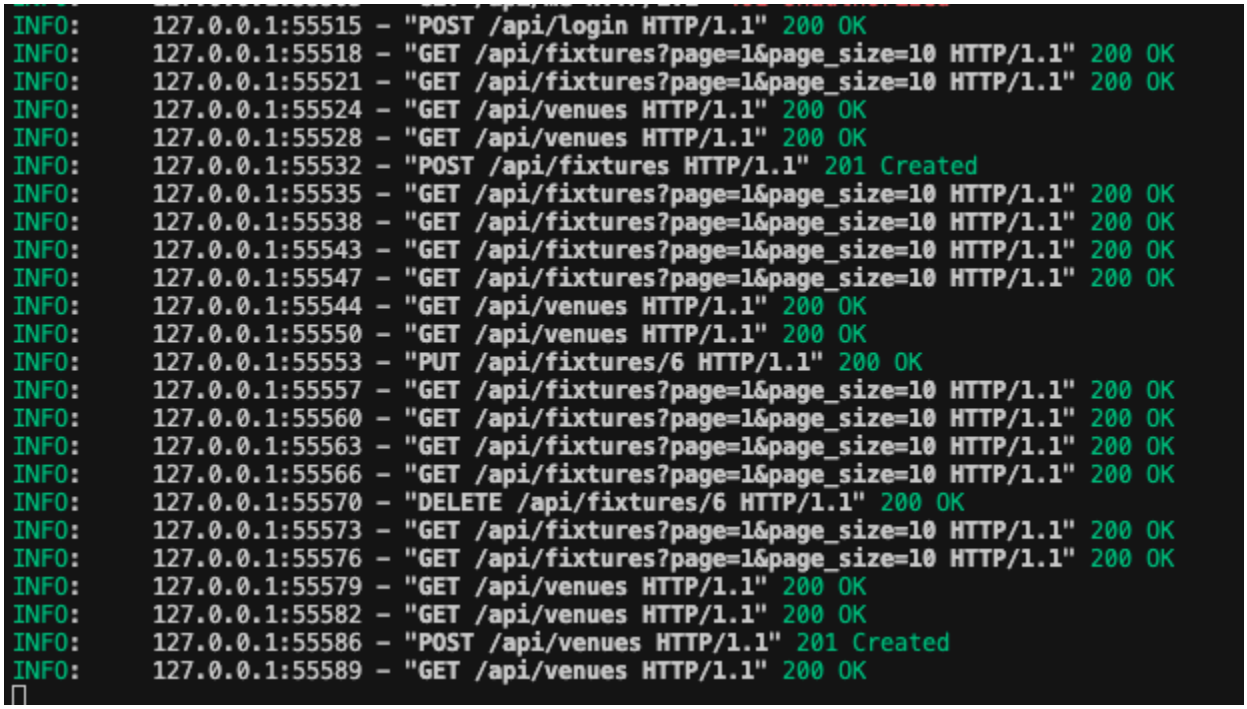
    return fixture_response(result)
```

Ananya Yallapragada
019142383

```python
@router.delete("/{fixture_id}", status_code=204)
def delete_fixture(fixture_id: str):
    try:
        result = fixtures_collection.delete_one(
            {"_id": fixture_object_id(fixture_id)}
        )
    except PyMongoError:
        raise HTTPException(
            status_code=500,
            detail="Failed to delete fixture",
        )

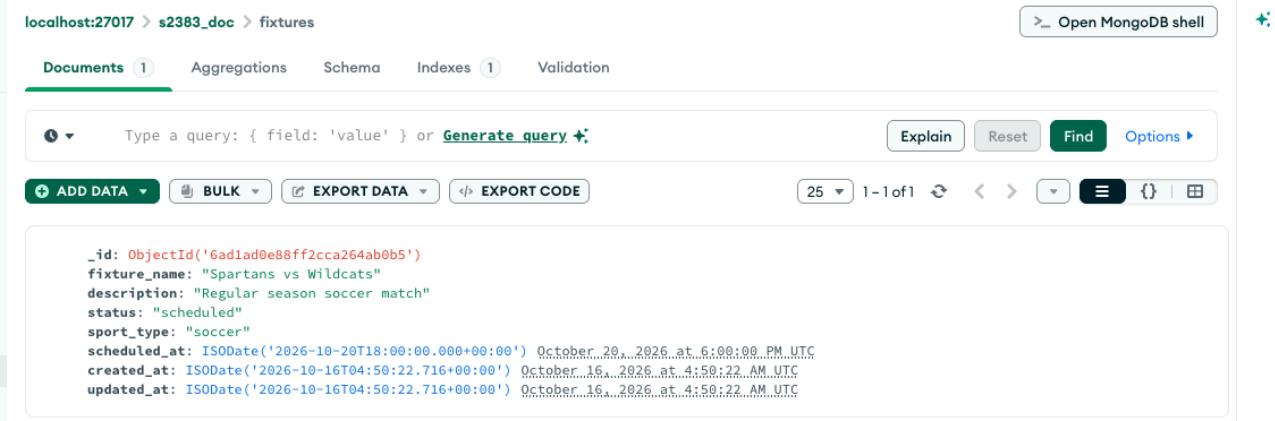
    if result.deleted_count == 0:
        raise HTTPException(status_code=404, detail="Fixture not found")
```

Terminal output: All CRUD operations - Successful



Ananya Yallapragada
019142383

On Compass:



III. Data Validation

Code snippets already pasted in the Database section.

Output:

Missing required fields:

```
(.venv) annie@mac web_application % curl -sS -X POST http://127.0.0.1:8583/api/documents
\
  -H "Content-Type: application/json" \
  -d '{"description":"Missing required fields"}' \
  -o /tmp/hw6_missing_fields.json \
  -w "Missing required fields: HTTP %{http_code}\n"

../../.venv/bin/python -m json.tool /tmp/hw6_missing_fields.json
Missing required fields: HTTP 422
{
    "detail": [
        {
            "type": "missing",
            "loc": [
                "body",
                "fixture_name"
            ],
            "msg": "Field required",
            "input": {
                "description": "Missing required fields"
            }
        },
        {
            "type": "missing",
            "loc": [
                "body",
                "sport_type"
            ],
            "msg": "Field required",
            "input": {
                "description": "Missing required fields"
            }
        },
        {
            "type": "missing",
            "loc": [
                "body",
                "scheduled_at"
            ],
            "msg": "Field required",
            "input": {
                "description": "Missing required fields"
            }
        }
    ]
}
```

Ananya Yallapragada
019142383

Invalid enum value:



```
(.venv) annie@mac web_application % curl -sS -X POST http://127.0.0.1:8583/api/documents
\
  -H "Content-Type: application/json" \
  -d '{"fixture_name":"Invalid sport","sport_type":"cricket","scheduled_at":"2026-11-20T1
8:00:00Z"}' \
  -o /tmp/hw6_invalid_enum.json \
  -w "Invalid enum: HTTP %{http_code}\n"

../../.venv/bin/python -m json.tool /tmp/hw6_invalid_enum.json
Invalid enum: HTTP 422
{
    "detail": [
        {
            "type": "enum",
            "loc": [
                "body",
                "sport_type"
            ],
            "msg": "Input should be 'soccer', 'basketball', 'tennis' or 'pickleball'",
            "input": "cricket",
            "ctx": {
                "expected": "'soccer', 'basketball', 'tennis' or 'pickleball'"
            }
        }
    ]
}
```

Incorrect data type:



```
(.venv) annie@mac web_application % curl -sS -X POST http://127.0.0.1:8583/api/documents
\
  -H "Content-Type: application/json" \
  -d '{"fixture_name":123,"sport_type":"tennis","scheduled_at":"2026-11-20T18:00:00Z"}' \

  -o /tmp/hw6_invalid_type.json \
  -w "Incorrect type: HTTP %{http_code}\n"

../../.venv/bin/python -m json.tool /tmp/hw6_invalid_type.json
Incorrect type: HTTP 422
{
    "detail": [
        {
            "type": "string_type",
            "loc": [
                "body",
                "fixture_name"
            ],
            "msg": "Input should be a valid string",
            "input": 123
        }
    ]
}
```

Ananya Yallapragada
019142383

Part 2 - Short-, Long-, and Episodic Memory

code/web_application/routers/memory.py

```python
@router.get("/memory-config")
def get_memory_config():
    return {
        "short_term_n": SHORT_TERM_N,
        "compact_short_term_n": COMPACT_SHORT_TERM_N,
    }


@router.post("/chat", response_model=ChatResponse)
def chat(request: ChatRequest):
    session_id = request.session_id or request.user_id
    return generate_reply(
        request.user_id,
        session_id,
        request.message,
        experiment_id=request.experiment_id,
        experiment_phase=request.experiment_phase,
    )


@router.get("/memory/{user_id}")
def get_memory(user_id: str):
    return fetch_memory_snapshot(user_id)


@router.get("/aggregate/{user_id}")
def get_aggregate(user_id: str):
    return aggregate_user_memory(user_id)
```

Ananya Yallapragada
019142383

Output:

```
annie@mac web_application % echo "Name: Ananya"
date
git -C ../.. rev-parse --short HEAD
ollama list
Name: Ananya
Mon Oct 19 01:26:50 PDT 2026
6a62c7c
NAME                       ID              SIZE      MODIFIED
nomic-embed-text:latest    0a109f422b47    274 MB    2 weeks ago
qwen3:8b                   500a1f067a9f    5.2 GB    7 weeks ago
annie@mac web_application % chat() {
  curl -sS -X POST http://127.0.0.1:8583/api/chat \
    -H "Content-Type: application/json" \
    -d "{\"user_id\":\"hw6-demo-user\",\"session_id\":\"hw6-demo-session\",\"message\":\"$1\"}" \
    | ../../.venv/bin/python -c 'import json, sys; print("Assistant:", json.load(sys.stdin)["reply"])'
}
```

```
annie@mac web_application % chat "My name is Ananya, and I organize a community sports league."
chat "I prefer tennis matches on weekday evenings."
chat "My preferred venue is Spartan Tennis Complex."
chat "I am planning a pickleball fundraiser on November 20."
chat "Please remember that players need to bring water."
chat "I want the fundraiser to start at 6 PM."
chat "My favorite color for event flyers is blue."
chat "I need the tennis schedule to avoid Monday evenings."
chat "I expect about 40 attendees at the fundraiser."
chat "Can you summarize my preferences and plans?"
Assistant: Hello, Ananya! It's great to meet you. As someone who organizes a community sports league, I'm here to help with
any questions or needs you might have. How can I assist you today?
Assistant: Thank you for sharing, Ananya! It’s great to know your preference for weekday evening tennis matches. How can I a
ssist you with scheduling or organizing these matches?
Assistant: Thank you for specifying, Ananya! The Spartan Tennis Complex is an excellent choice for your weekday evening tenn
is matches. Would you like assistance with scheduling or confirming availability at the venue?
Assistant: Thank you for sharing, Ananya! Your pickleball fundraiser on November 20 is noted. Would you like assistance with
 scheduling the event or confirming availability at the Spartan Tennis Complex? Let me know how I can help!
Assistant: Thank you for the reminder, Ananya! I've noted that players need to bring water for both your pickleball fundrais
er on November 20 and your weekday evening tennis matches at the Spartan Tennis Complex. Would you like assistance with sche
duling, confirming venue availability, or anything else? Let me know how I can help!
Assistant: Thank you for confirming, Ananya! Your pickleball fundraiser is scheduled for **November 20 at 6 PM** at the **Sp
artan Tennis Complex**. Players will need to bring water, as noted. Would you like assistance with finalizing details, confi
rming availability, or anything else? Let me know!
Assistant: Thank you for sharing, Ananya! I've noted your preference for **blue** as the favorite color for event flyers. Wo
uld you like assistance with designing flyers, confirming event details, or anything else? Let me know how I can help!
Assistant: Thank you for the clarification, Ananya! To avoid Monday evenings for tennis matches, I’ll ensure the schedule is
 set for **Tuesday–Friday evenings** at the **Spartan Tennis Complex**. Your pickleball fundraiser is already confirmed for
**November 20 at 6 PM**. Would you like to finalize the tennis match dates or confirm venue availability? Let me know how I
can assist! 🎾💙
Assistant: Thank you for the update, Ananya! With **40 attendees** expected at your pickleball fundraiser on **November 20 a
t 6 PM** at the **Spartan Tennis Complex**, I’ll ensure the event is prepared accordingly. Would you like assistance with fi
nalizing logistics, confirming venue details, or anything else? Let me know how I can help! 🎾💙
Assistant: Here’s a summary of your preferences and plans:

### **Tennis Matches**
- **Preferred days**: Weekday evenings (Tuesday–Friday), **avoiding Monday evenings**.
- **Venue**: **Spartan Tennis Complex** for all matches.
- **Logistics**: All players must bring **water**.

### **Pickleball Fundraiser**
- **Date/Time**: **November 20 at 6 PM**.
- **Venue**: **Spartan Tennis Complex**.
- **Attendees**: Expected **40 people**.
- **Flyer Design**: Preferred color is **blue**.

Let me know if you’d like to confirm details or proceed with next steps! 🎾💙
```

Ananya Yallapragada
019142383

code/web_application/memory_service.py

```python
def extract_and_store_episodes(user_id: str, session_id: str, message: str) -> None:
    extraction_prompt = [
        {
            "role": "system",
            "content": (
                "Extract up to three durable user facts, preferences, or commitments. "
                "Return only a JSON array with objects containing fact and importance "
                "(a number from 0 to 1). Preserve stated details exactly and do not "
                "infer missing details. Return [] when there are no useful facts."
            ),
        },
        {"role": "user", "content": message},
    ]
    facts = parse_facts(model_text(extraction_prompt, max_tokens=120))

    new_facts = [
        fact
        for fact in facts
        if not episode_exists(user_id, session_id, fact["fact"])
    ]
    if not new_facts:
        return

    embeddings = embed_texts([fact["fact"] for fact in new_facts])
    for fact, embedding in zip(new_facts, embeddings):
        save_episode(
            user_id,
            session_id,
            fact["fact"],
            fact["importance"],
            embedding,
        )
```

```python
def retrieve_episodes(user_id: str, message: str) -> list[dict]:
    query_embedding = embed_texts([message])[0]
    scored_episodes = []

    for episode in fetch_user_episodes(user_id):
        score = cosine_similarity(query_embedding, episode["embedding"])
        scored_episodes.append((score, episode))

    scored_episodes.sort(key=lambda item: item[0], reverse=True)
    return [episode for _, episode in scored_episodes[:TOP_K_EPISODES]]
```

```python
def cosine_similarity(left: list[float], right: list[float]) -> float:
    if len(left) != len(right) or not left:
        return 0.0

    dot_product = sum(a * b for a, b in zip(left, right))
    left_norm = math.sqrt(sum(value * value for value in left))
    right_norm = math.sqrt(sum(value * value for value in right))

    if left_norm == 0 or right_norm == 0:
        return 0.0

    return dot_product / (left_norm * right_norm)
```

Ananya Yallapragada
019142383

Output:



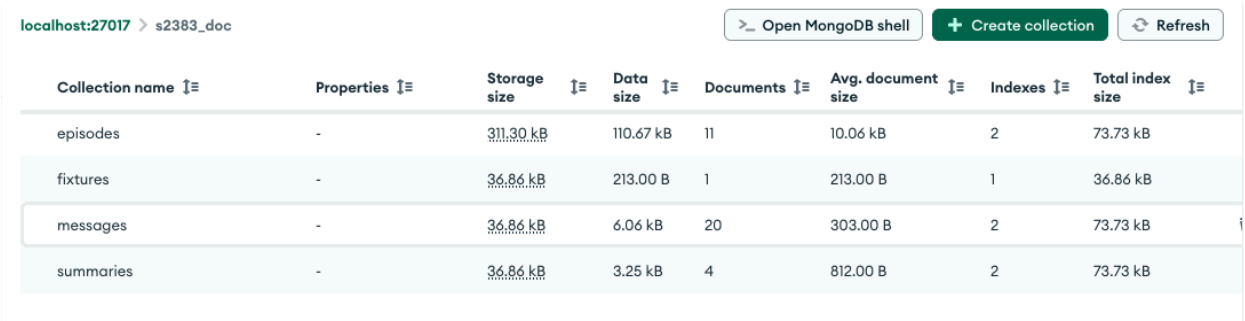
```
annie@mac web_application % curl -sS http://127.0.0.1:8583/api/aggregate/hw6-demo-user \
  -o ../../reports/hw06/raw/aggregate_demo.json

../../.venv/bin/python -m json.tool \
  ../../reports/hw06/raw/aggregate_demo.json
{
    "daily_message_counts": [
        {
            "message_count": 20,
            "date": "2026-10-19"
        }
    ],
    "recent_summaries": [
        {
            "user_id": "hw6-demo-user",
            "session_id": null,
            "scope": "user",
            "text": "**Ananya's Lifetime User Summary**  \n**Preferences**:  \n- Tennis matches preferred on **weekday eveni
ngs (Tuesday\u2013Friday)**, avoiding **Monday evenings**.  \n- **Preferred venue**: **Spartan Tennis Complex** for all tenn
is-related events.  \n- **Players must bring water** for all events.  \n- **Flyer color preference**: **Blue**.  \n\n**Goals
/Decisions**:  \n- Organize community sports league with **weekday evening tennis matches**.  \n- Plan **pickleball fundrais
er** on **November 20 at 6 PM** at **Spartan Tennis Complex**.  \n\n**Confirmed Details**:  \n- **Pickleball Fundraiser**: *
*November 20 at 6 PM** at **Spartan Tennis Complex**.  \n- **Expected attendees**: **40**.  \n- **No conflicts** or addition
al requirements noted.  \n\n**Event Logistics**:  \n- **Water requirement**: All attendees must bring water for both tennis
matches and the pickleball fundraiser.  \n- **Venue consistency**: **Spartan Tennis Complex** for both tennis matches and th
e fundraiser.  \n\n**Next Steps**:  \n- Confirm **Spartan Tennis Complex availability** for tennis matches and the fundraise
r.  \n- Finalize **weekday evening tennis schedule**.  \n- Prepare **blue-themed flyers** and ensure logistics (water, venue
) are set.  \n\n**No conflicts or missing details inferred.**",
            "created_at": "2026-10-19T08:36:09.159000"
        },
        {
            "user_id": "hw6-demo-user",
            "session_id": "hw6-demo-session",
            "scope": "session",
            "text": "- **Tennis Matches**: Prefer weekday evenings (Tuesday\u2013Friday) at **Spartan Tennis Complex**; avoi
d Monday evenings.  \n- **Pickleball Fundraiser**: Scheduled for **November 20 at 6 PM** at **Spartan Tennis Complex**.  \n-
 **Event Logistics**: Players must bring water for both events.  \n- **Flyer Design**: Preferred color for flyers is **blue*
*.  \n- **Attendees**: Expect **40 attendees** at the pickleball fundraiser.  \n- **Next Steps**: Confirm venue availability
, finalize tennis schedule, and prepare event logistics.  \n\nNo conflicts noted.",
            "created_at": "2026-10-19T08:35:01.149000"
        },
        {
            "user_id": "hw6-demo-user",
            "session_id": "hw6-demo-session",
            "scope": "session",
            "text": "- **Goals/Decisions**:  \n  - Organize community sports league with tennis matches on weekday evenings.
  \n  - Plan pickleball fundraiser on **November 20** at **6 PM**.  \n\n- **Preferences/Commitments**:  \n  - **Preferred ve
nue**: Spartan Tennis Complex for both tennis matches and the pickleball fundraiser.  \n  - **Players must bring water** for
 all events.  \n\n- **Confirmed Details**:  \n  - Fundraiser: **November 20 at 6 PM** at **Spartan Tennis Complex**.  \n  -
No conflicts or additional requirements noted.  \n\n- **Next Steps**:  \n  - Assistance requested for scheduling, confirming
 venue availability, or finalizing details.",
            "created_at": "2026-10-19T08:32:31.003000"
        },
        {
            "user_id": "hw6-demo-user",
            "session_id": "hw6-demo-session",
            "scope": "session",
            "text": "- **Ananya's Preference**: Tennis matches on weekday evenings.  \n- **Preferred Venue**: Spartan Tennis
 Complex.  \n- **Assistant's Role**: Offer assistance with scheduling/organizing matches or confirming venue availability.
\n- **No Conflicts**: All stated preferences and commitments are preserved without alteration.",
            "created_at": "2026-10-19T08:29:13.229000"
        }
    ]
}
```

memory_demo.json: Too long to capture a screenshot. Saved as json in repo.
Same with *aggregate_demo.json.* Both are in *reports/hw06/raw.*

Ananya Yallapragada
019142383

Compass with populated collections:



Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
episodes	-	311.30 kB	110.67 kB	11	10.06 kB	2	73.73 kB
fixtures	-	36.86 kB	213.00 B	1	213.00 B	1	36.86 kB
messages	-	36.86 kB	6.06 kB	20	303.00 B	2	73.73 kB
summaries	-	36.86 kB	3.25 kB	4	812.00 B	2	73.73 kB

MongoDB persistence

Code: code/web_application/memory_repository.py

```python
def save_message(
    user_id: str,
    session_id: str,
    role: Literal["user", "assistant"],
    content: str,
) -> None:
    messages_collection.insert_one(
        {
            "user_id": user_id,
            "session_id": session_id,
            "role": role,
            "content": content,
            "created_at": datetime.now(timezone.utc),
        }
    )
```

```python
def fetch_recent_session_messages(session_id: str, limit: int) -> list[dict]:
    messages = list(
        messages_collection.find(
            {"session_id": session_id},
            {"_id": 0},
        )
        .sort("created_at", -1)
        .limit(limit)
    )
    return list(reversed(messages))
```

Ananya Yallapragada
019142383

```python
def latest_summary(
    user_id: str,
    session_id: str | None,
    scope: Literal["session", "user"],
) -> dict | None:
    return summaries_collection.find_one(
        {
            "user_id": user_id,
            "session_id": session_id,
            "scope": scope,
        },
        {"_id": 0},
        sort=[("created_at", -1)],
    )


def latest_session_summary(user_id: str) -> dict | None:
    return summaries_collection.find_one(
        {"user_id": user_id, "scope": "session"},
        {"_id": 0},
        sort=[("created_at", -1)],
    )
```

```python
def save_episode(
    user_id: str,
    session_id: str,
    fact: str,
    importance: float,
    embedding: list[float],
) -> None:
    episodes_collection.insert_one(
        {
            "user_id": user_id,
            "session_id": session_id,
            "fact": fact,
            "importance": float(importance),
            "embedding": [float(value) for value in embedding],
            "created_at": datetime.now(timezone.utc),
        }
    )
```

```python
def fetch_user_episodes(user_id: str) -> list[dict]:
    return list(
        episodes_collection.find(
            {"user_id": user_id},
            {"_id": 0},
        )
    )
```

Ananya Yallapragada
019142383

Part 3 - What Memory Costs

1. *Metrics logging: memory_service.py*

```python
def generate_reply(
    user_id: str,
    session_id: str,
    message: str,
    experiment_id: str | None = None,
    experiment_phase: str | None = None,
) -> dict:
    started_at = time.perf_counter()
    save_message(user_id, session_id, "user", message)
    summary = long_term_memory(user_id, session_id)
    short_term_limit = effective_short_term_n(summary)
    short_term_messages = fetch_recent_session_messages(
        session_id,
        short_term_limit + 1,
    )[:-1]
    extract_and_store_episodes(user_id, session_id, message)
    episodic_facts = retrieve_episodes(user_id, message)
    prompt = build_prompt(short_term_messages, summary, episodic_facts, message)
    reply_response = complete(
        prompt,
        model=CHAT_MODEL,
        temperature=0.2,
        max_tokens=180,
    )
    reply = str(reply_response.content).strip()
    token_usage = get_token_usage(reply_response)
    component_tokens = prompt_component_tokens(
        short_term_messages,
        summary,
        episodic_facts,
    )
    save_message(user_id, session_id, "assistant", reply)
    maybe_update_summaries(user_id, session_id)

    append_chat_metric(
        {
```

2. *Compression policy - memory_service.py*

```python
def effective_short_term_n(long_term_summary: str) -> int:
    if COMPACT_SHORT_TERM_N > 0 and long_term_summary:
        return min(SHORT_TERM_N, COMPACT_SHORT_TERM_N)
    return SHORT_TERM_N
```

Ananya Yallapragada
019142383

3. *Fixed probes and rubric: memory_probes.yaml*

```yaml
version: 1
conversation_id: hw6-fixed-conversation
scoring_rubric:
  5: Meets every expected criterion without unsupported claims.
  4: Meets the main criteria with one minor omission.
  3: Meets some criteria but omits or weakens important information.
  2: Mostly incorrect, incomplete, or includes unsupported claims.
  1: Incorrect, irrelevant, or fails to recall the required information.
probes:
  - id: identity_and_role
    question: What is my name and what do I organize?
    expected_answer_criteria:
      - Identifies the user as Ananya.
      - States that Ananya organizes a community sports league.
  - id: tennis_preferences
    question: When do I prefer tennis matches, and what time should be avoided?
    expected_answer_criteria:
      - States a preference for weekday evenings.
      - States that Monday evenings should be avoided.
  - id: fundraiser_details
    question: What are the date, time, and venue for the pickleball fundraiser?
    expected_answer_criteria:
      - States November 20.
      - States 6 PM.
      - States Spartan Tennis Complex.
  - id: fundraiser_logistics
    question: What logistics and design details should I remember for the fundraiser?
    expected_answer_criteria:
      - States that players need to bring water.
      - States that flyers should be blue.
      - States that about 40 attendees are expected.
  - id: memory_boundaries
    question: Which facts are about tennis scheduling, and which are about the fundraiser?
    expected_answer_criteria:
      - Separates weekday-evening and avoid-Monday preferences as tennis scheduling facts.
      - Separates November 20, 6 PM, and 40 attendees as fundraiser facts.
      - Does not invent an unprovided date, time, or venue.
```

probe_scores.json U

reports > hw06 > raw > {} probe_scores.json > {} runs > {} stn4 > {} fundraiser_logistics > {} matched_criteria

```json
{
  "scoring_method": "Deterministic phrase checks against the expected criteria in memory_pro
  "runs": {
    "stn4": {
      "identity_and_role": {
        "score": 5,
        "matched_criteria": {
          "name": true,
          "role": true
        }
      },
      "tennis_preferences": {
        "score": 5,
        "matched_criteria": {
          "weekday evenings": true,
          "avoid Monday evenings": true
        }
      },
      "fundraiser_details": {
        "score": 5,
        "matched_criteria": {
          "November 20": true,
          "6 PM": true,
          "Spartan Tennis Complex": true
        }
      },
      "fundraiser_logistics": {
        "score": 4,
        "matched_criteria": {
          "bring water": true,
          "blue flyers": true,
          "about 40 attendees": false
        }
      },
      "memory_boundaries": {
        "score": 4,
```

Ananya Yallapragada
019142383

4. *Reproducible experiment and analysis:*

run_memory_experiment.py

```
try:
    for turn, message in enumerate(CONVERSATION, start=1):
        response = post_chat(
            arguments.base_url,
            user_id,
            session_id,
            message,
            arguments.run_id,
            "conversation",
            arguments.timeout,
        )
        results["conversation"].append(
            {"turn": turn, "message": message, "reply": response["reply"]}
        )
        save_results(results, arguments.output)
        print(f"Conversation turn {turn}/30 complete")

    for probe in probes:
        response = post_chat(
            arguments.base_url,
            user_id,
            session_id,
            probe["question"],
            arguments.run_id,
            "probe",
            arguments.timeout,
        )
        results["probes"].append(
            {
                "id": probe["id"],
                "question": probe["question"],
                "expected_answer_criteria": probe["expected_answer_criteria"],
                "reply": response["reply"],
            }
        )
        save_results(results, arguments.output)
        print(f"Probe {probe['id']} complete")
except requests.RequestException as error:
    results["status"] = "failed"
    results["error"] = str(error)
    results["completed_at"] = time.time()
    save_results(results, arguments.output)
    raise SystemExit(f"Run stopped: {error}") from error
```

analyze_memory_experiments.py

```
def score_reply(reply: str, rules: dict[str, tuple[str, ...]]) -> dict:
    text = re.sub(r"[*_`#]", "", reply.casefold())
    matched = {
        criterion: any(re.search(pattern, text) for pattern in patterns)
        for criterion, patterns in rules.items()
    }
    count = sum(matched.values())
    total = len(matched)
    if count == total:
        score = 5
    elif count == total - 1:
        score = 4
    elif count >= (total + 1) // 2:
        score = 3
    elif count:
        score = 2
    else:
        score = 1
    return {"score": score, "matched_criteria": matched}
```

Ananya Yallapragada
019142383

Terminal Outputs:

```
(.venv) annie@mac web_application % date
git -C ../.. rev-parse --short HEAD

../../.venv/bin/python run_memory_experiment.py \
  --short-term-n 4 \
  --compact-short-term-n 0 \
  --run-id stn4 \
  --output ../../reports/hw06/raw/experiment_n4.json

Mon Oct 19 08:27:01 PDT 2026
5f15bc7
Conversation turn 1/30 complete
Conversation turn 2/30 complete
Conversation turn 3/30 complete
Conversation turn 4/30 complete
Conversation turn 5/30 complete
Conversation turn 6/30 complete
Conversation turn 7/30 complete
Conversation turn 8/30 complete
Conversation turn 9/30 complete
Conversation turn 10/30 complete
Conversation turn 11/30 complete
Conversation turn 12/30 complete
Conversation turn 13/30 complete
Conversation turn 14/30 complete
Conversation turn 15/30 complete
Conversation turn 16/30 complete
Conversation turn 17/30 complete
Conversation turn 18/30 complete
Conversation turn 19/30 complete
Conversation turn 20/30 complete
Conversation turn 21/30 complete
Conversation turn 22/30 complete
Conversation turn 23/30 complete
Conversation turn 24/30 complete
Conversation turn 25/30 complete
Conversation turn 26/30 complete
Conversation turn 27/30 complete
Conversation turn 28/30 complete
Conversation turn 29/30 complete
Conversation turn 30/30 complete
Probe identity_and_role complete
Probe tennis_preferences complete
Probe fundraiser_details complete
Probe fundraiser_logistics complete
Probe memory_boundaries complete
Saved results to ../../reports/hw06/raw/experiment_n4.json
(.venv) annie@mac web_application %
```

```
(.venv) annie@mac web_application % date
git -C ../.. rev-parse --short HEAD
echo "SHORT_TERM_N=12 COMPACT_SHORT_TERM_N=0"

../../.venv/bin/python run_memory_experiment.py \
  --short-term-n 12 \
  --compact-short-term-n 0 \
  --run-id stn12 \
  --output ../../reports/hw06/raw/experiment_n12.json
Mon Oct 19 08:58:08 PDT 2026
5f15bc7
SHORT_TERM_N=12 COMPACT_SHORT_TERM_N=0
Conversation turn 1/30 complete
Conversation turn 2/30 complete
Conversation turn 3/30 complete
Conversation turn 4/30 complete
Conversation turn 5/30 complete
Conversation turn 6/30 complete
Conversation turn 7/30 complete
Conversation turn 8/30 complete
Conversation turn 9/30 complete
Conversation turn 10/30 complete
Conversation turn 11/30 complete
Conversation turn 12/30 complete
Conversation turn 13/30 complete
Conversation turn 14/30 complete
Conversation turn 15/30 complete
Conversation turn 16/30 complete
Conversation turn 17/30 complete
Conversation turn 18/30 complete
Conversation turn 19/30 complete
Conversation turn 20/30 complete
Conversation turn 21/30 complete
Conversation turn 22/30 complete
Conversation turn 23/30 complete
Conversation turn 24/30 complete
Conversation turn 25/30 complete
Conversation turn 26/30 complete
Conversation turn 27/30 complete
Conversation turn 28/30 complete
Conversation turn 29/30 complete
Conversation turn 30/30 complete
Probe identity_and_role complete
Probe tennis_preferences complete
Probe fundraiser_details complete
Probe fundraiser_logistics complete
Probe memory_boundaries complete
Saved results to ../../reports/hw06/raw/experiment_n12.json
```

```
(.venv) annie@mac web_application % date
git -C ../.. rev-parse --short HEAD
echo "SHORT_TERM_N=30 COMPACT_SHORT_TERM_N=0"

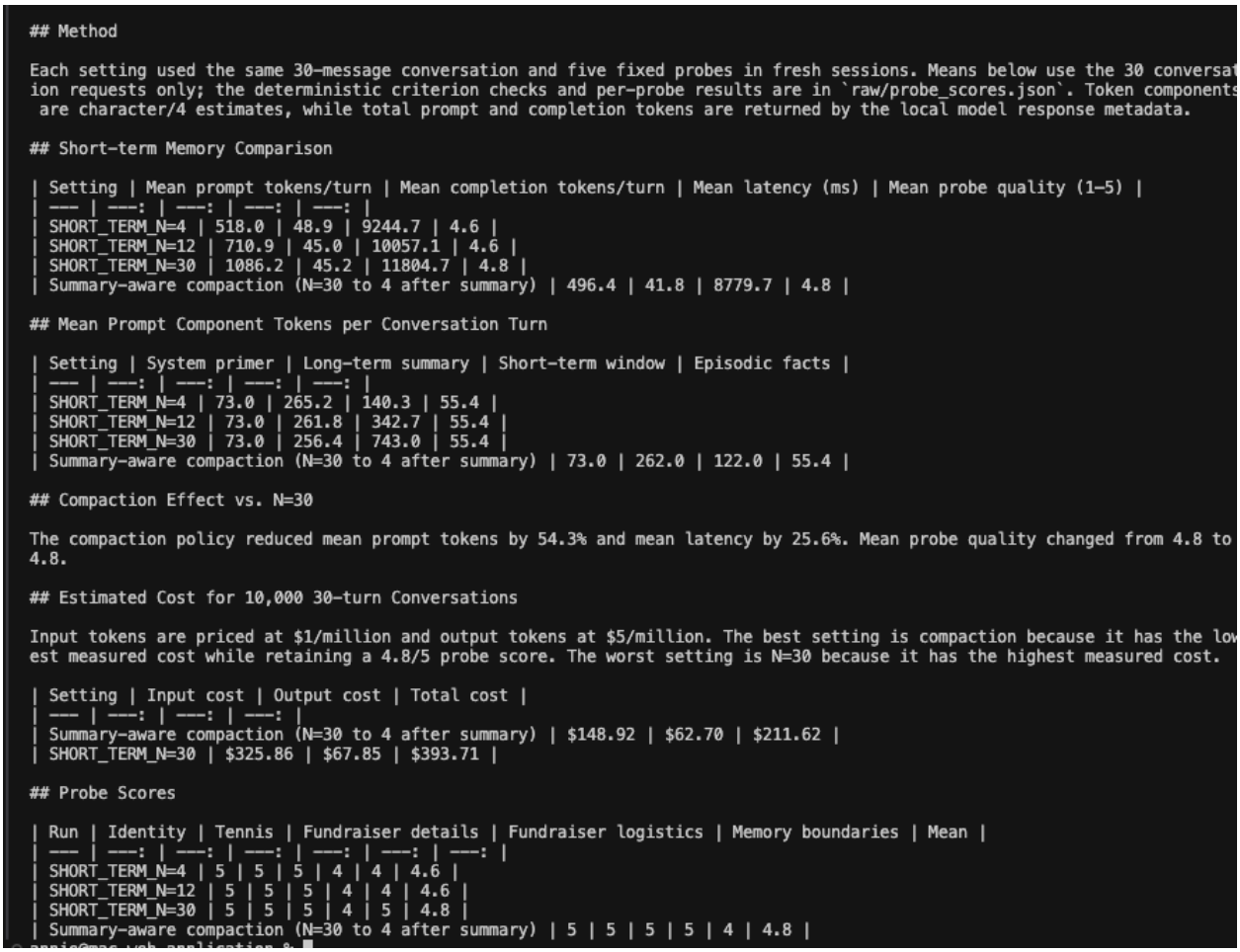
../../.venv/bin/python run_memory_experiment.py \
  --short-term-n 30 \
  --compact-short-term-n 0 \
  --run-id stn30 \
  --output ../../reports/hw06/raw/experiment_n30.json
Mon Oct 19 09:09:22 PDT 2026
5f15bc7
SHORT_TERM_N=30 COMPACT_SHORT_TERM_N=0
Conversation turn 1/30 complete
Conversation turn 2/30 complete
Conversation turn 3/30 complete
Conversation turn 4/30 complete
Conversation turn 5/30 complete
Conversation turn 6/30 complete
Conversation turn 7/30 complete
Conversation turn 8/30 complete
Conversation turn 9/30 complete
Conversation turn 10/30 complete
Conversation turn 11/30 complete
Conversation turn 12/30 complete
Conversation turn 13/30 complete
Conversation turn 14/30 complete
Conversation turn 15/30 complete
Conversation turn 16/30 complete
Conversation turn 17/30 complete
Conversation turn 18/30 complete
Conversation turn 19/30 complete
Conversation turn 20/30 complete
Conversation turn 21/30 complete
Conversation turn 22/30 complete
Conversation turn 23/30 complete
Conversation turn 24/30 complete
Conversation turn 25/30 complete
Conversation turn 26/30 complete
Conversation turn 27/30 complete
Conversation turn 28/30 complete
Conversation turn 29/30 complete
Conversation turn 30/30 complete
Probe identity_and_role complete
Probe tennis_preferences complete
Probe fundraiser_details complete
Probe fundraiser_logistics complete
Probe memory_boundaries complete
Saved results to ../../reports/hw06/raw/experiment_n30.json
```

```
(.venv) annie@mac web_application % date
git -C ../.. rev-parse --short HEAD
echo "SHORT_TERM_N=30 COMPACT_SHORT_TERM_N=4"

../../.venv/bin/python run_memory_experiment.py \
  --short-term-n 30 \
  --compact-short-term-n 4 \
  --run-id compaction \
  --output ../../reports/hw06/raw/experiment_compaction.json
Mon Oct 19 09:23:40 PDT 2026
5f15bc7
SHORT_TERM_N=30 COMPACT_SHORT_TERM_N=4
Conversation turn 1/30 complete
Conversation turn 2/30 complete
Conversation turn 3/30 complete
Conversation turn 4/30 complete
Conversation turn 5/30 complete
Conversation turn 6/30 complete
Conversation turn 7/30 complete
Conversation turn 8/30 complete
Conversation turn 9/30 complete
Conversation turn 10/30 complete
Conversation turn 11/30 complete
Conversation turn 12/30 complete
Conversation turn 13/30 complete
Conversation turn 14/30 complete
Conversation turn 15/30 complete
Conversation turn 16/30 complete
Conversation turn 17/30 complete
Conversation turn 18/30 complete
Conversation turn 19/30 complete
Conversation turn 20/30 complete
Conversation turn 21/30 complete
Conversation turn 22/30 complete
Conversation turn 23/30 complete
Conversation turn 24/30 complete
Conversation turn 25/30 complete
Conversation turn 26/30 complete
Conversation turn 27/30 complete
Conversation turn 28/30 complete
Conversation turn 29/30 complete
Conversation turn 30/30 complete
Probe identity_and_role complete
Probe tennis_preferences complete
Probe fundraiser_details complete
Probe fundraiser_logistics complete
Probe memory_boundaries complete
Saved results to ../../reports/hw06/raw/experiment_compaction.json
```

Ananya Yallapragada
019142383

Analysis compiled in Metrics.md (printed on terminal)



```
## Method

Each setting used the same 30-message conversation and five fixed probes in fresh sessions. Means below use the 30 conversat
ion requests only; the deterministic criterion checks and per-probe results are in `raw/probe_scores.json`. Token components
 are character/4 estimates, while total prompt and completion tokens are returned by the local model response metadata.

## Short-term Memory Comparison

| Setting | Mean prompt tokens/turn | Mean completion tokens/turn | Mean latency (ms) | Mean probe quality (1–5) |
| --- | ---: | ---: | ---: | ---: |
| SHORT_TERM_N=4 | 518.0 | 48.9 | 9244.7 | 4.6 |
| SHORT_TERM_N=12 | 710.9 | 45.0 | 10057.1 | 4.6 |
| SHORT_TERM_N=30 | 1086.2 | 45.2 | 11804.7 | 4.8 |
| Summary-aware compaction (N=30 to 4 after summary) | 496.4 | 41.8 | 8779.7 | 4.8 |

## Mean Prompt Component Tokens per Conversation Turn

| Setting | System primer | Long-term summary | Short-term window | Episodic facts |
| --- | ---: | ---: | ---: | ---: |
| SHORT_TERM_N=4 | 73.0 | 265.2 | 140.3 | 55.4 |
| SHORT_TERM_N=12 | 73.0 | 261.8 | 342.7 | 55.4 |
| SHORT_TERM_N=30 | 73.0 | 256.4 | 743.0 | 55.4 |
| Summary-aware compaction (N=30 to 4 after summary) | 73.0 | 262.0 | 122.0 | 55.4 |

## Compaction Effect vs. N=30

The compaction policy reduced mean prompt tokens by 54.3% and mean latency by 25.6%. Mean probe quality changed from 4.8 to
4.8.

## Estimated Cost for 10,000 30-turn Conversations

Input tokens are priced at $1/million and output tokens at $5/million. The best setting is compaction because it has the low
est measured cost while retaining a 4.8/5 probe score. The worst setting is N=30 because it has the highest measured cost.

| Setting | Input cost | Output cost | Total cost |
| --- | ---: | ---: | ---: |
| Summary-aware compaction (N=30 to 4 after summary) | $148.92 | $62.70 | $211.62 |
| SHORT_TERM_N=30 | $325.86 | $67.85 | $393.71 |

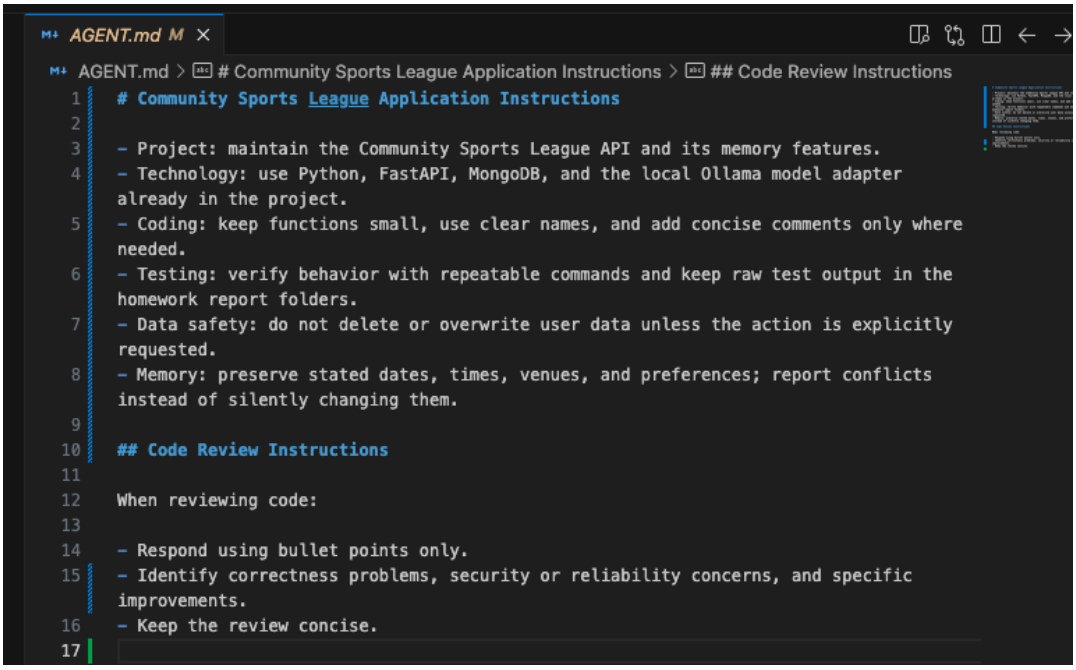
## Probe Scores

| Run | Identity | Tennis | Fundraiser details | Fundraiser logistics | Memory boundaries | Mean |
| --- | ---: | ---: | ---: | ---: | ---: | ---: |
| SHORT_TERM_N=4 | 5 | 5 | 5 | 4 | 4 | 4.6 |
| SHORT_TERM_N=12 | 5 | 5 | 5 | 4 | 4 | 4.6 |
| SHORT_TERM_N=30 | 5 | 5 | 5 | 4 | 5 | 4.8 |
| Summary-aware compaction (N=30 to 4 after summary) | 5 | 5 | 5 | 5 | 4 | 4.8 |
```

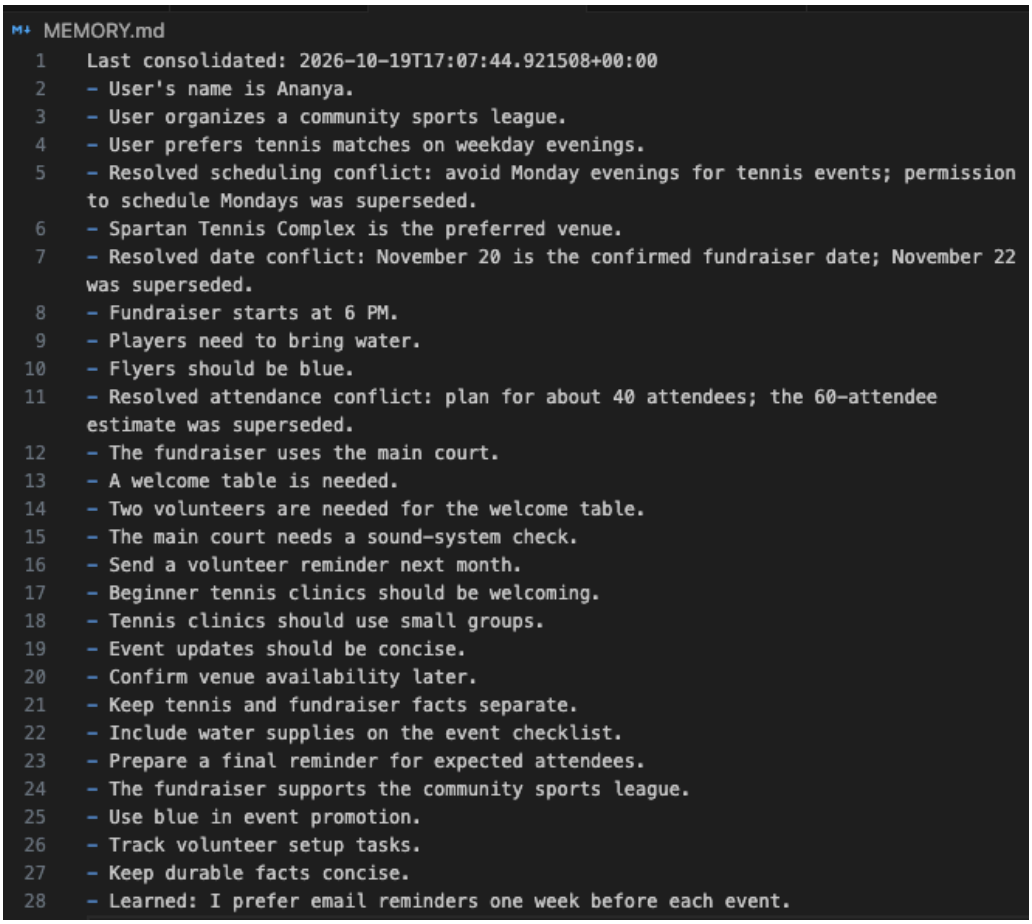
Ananya Yallapragada
019142383

Part 4 - File-based Memory and Consolidation

AGENT.md

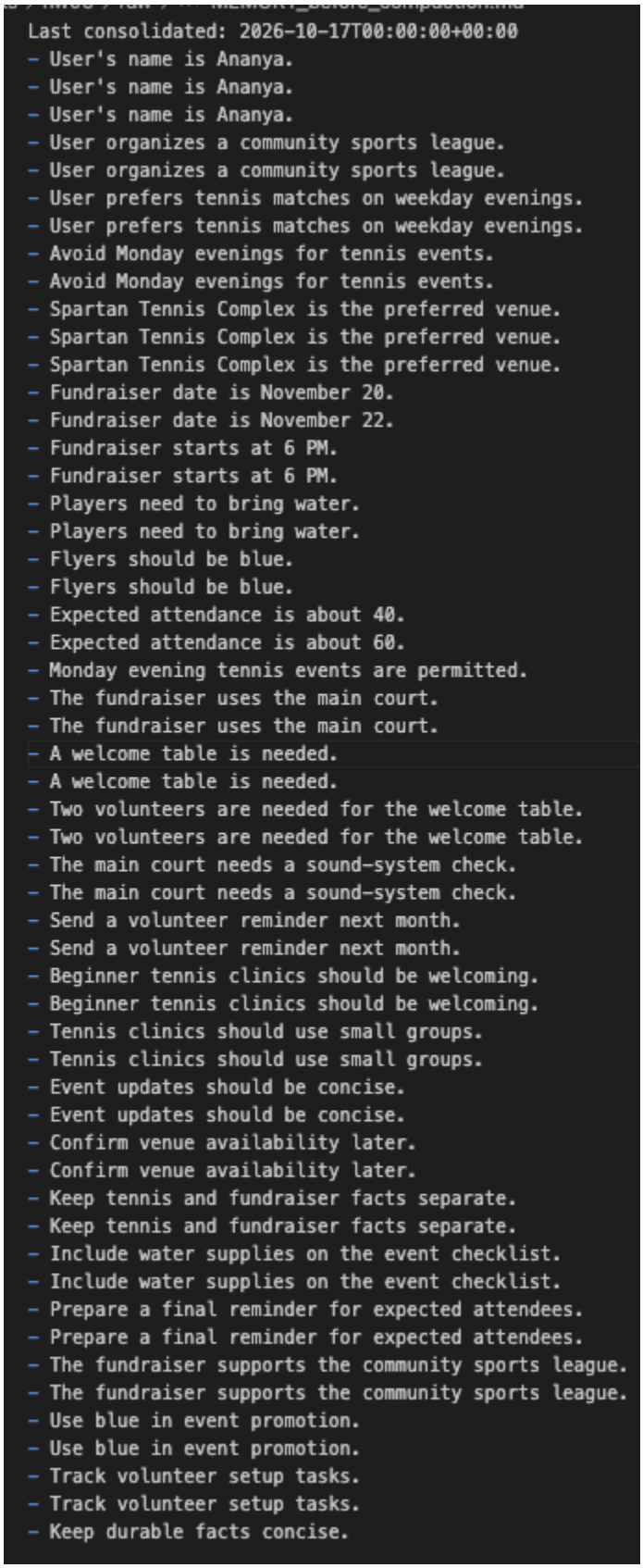


MEMORY.md



Ananya Yallapragada
019142383

MEMORY_before_compaction.md



```
Last consolidated: 2026-10-17T00:00:00+00:00
- User's name is Ananya.
- User's name is Ananya.
- User's name is Ananya.
- User organizes a community sports league.
- User organizes a community sports league.
- User prefers tennis matches on weekday evenings.
- User prefers tennis matches on weekday evenings.
- Avoid Monday evenings for tennis events.
- Avoid Monday evenings for tennis events.
- Spartan Tennis Complex is the preferred venue.
- Spartan Tennis Complex is the preferred venue.
- Spartan Tennis Complex is the preferred venue.
- Fundraiser date is November 20.
- Fundraiser date is November 22.
- Fundraiser starts at 6 PM.
- Fundraiser starts at 6 PM.
- Players need to bring water.
- Players need to bring water.
- Flyers should be blue.
- Flyers should be blue.
- Expected attendance is about 40.
- Expected attendance is about 60.
- Monday evening tennis events are permitted.
- The fundraiser uses the main court.
- The fundraiser uses the main court.
- A welcome table is needed.
- A welcome table is needed.
- Two volunteers are needed for the welcome table.
- Two volunteers are needed for the welcome table.
- The main court needs a sound-system check.
- The main court needs a sound-system check.
- Send a volunteer reminder next month.
- Send a volunteer reminder next month.
- Beginner tennis clinics should be welcoming.
- Beginner tennis clinics should be welcoming.
- Tennis clinics should use small groups.
- Tennis clinics should use small groups.
- Event updates should be concise.
- Event updates should be concise.
- Confirm venue availability later.
- Confirm venue availability later.
- Keep tennis and fundraiser facts separate.
- Keep tennis and fundraiser facts separate.
- Include water supplies on the event checklist.
- Include water supplies on the event checklist.
- Prepare a final reminder for expected attendees.
- Prepare a final reminder for expected attendees.
- The fundraiser supports the community sports league.
- The fundraiser supports the community sports league.
- Use blue in event promotion.
- Use blue in event promotion.
- Track volunteer setup tasks.
- Track volunteer setup tasks.
- Keep durable facts concise.
```

Ananya Yallapragada
019142383

file_memory_service.py

```python
def consolidation_status(now: datetime | None = None) -> dict:
    now = now or datetime.now(timezone.utc)
    consolidated_at = last_consolidated_at()
    age_hours = None if consolidated_at is None else (now - consolidated_at).total_seconds() / 3600
    line_count = len(fact_lines())
    return {
        "non_empty_fact_lines": line_count,
        "last_consolidated_at": consolidated_at.isoformat() if consolidated_at else None,
        "age_hours": round(age_hours, 2) if age_hours is not None else None,
        "needs_consolidation": line_count > 40 or age_hours is None or age_hours > 24,
    }


def build_file_memory_prompt(user_message: str) -> list[dict]:
    instructions = read_markdown(AGENT_PATH)
    memory = read_markdown(MEMORY_PATH)
    system_content = f"{instructions}\n\n# Durable Memory\n{memory}"
    return [
        {"role": "system", "content": system_content},
        {"role": "user", "content": user_message},
    ]
```

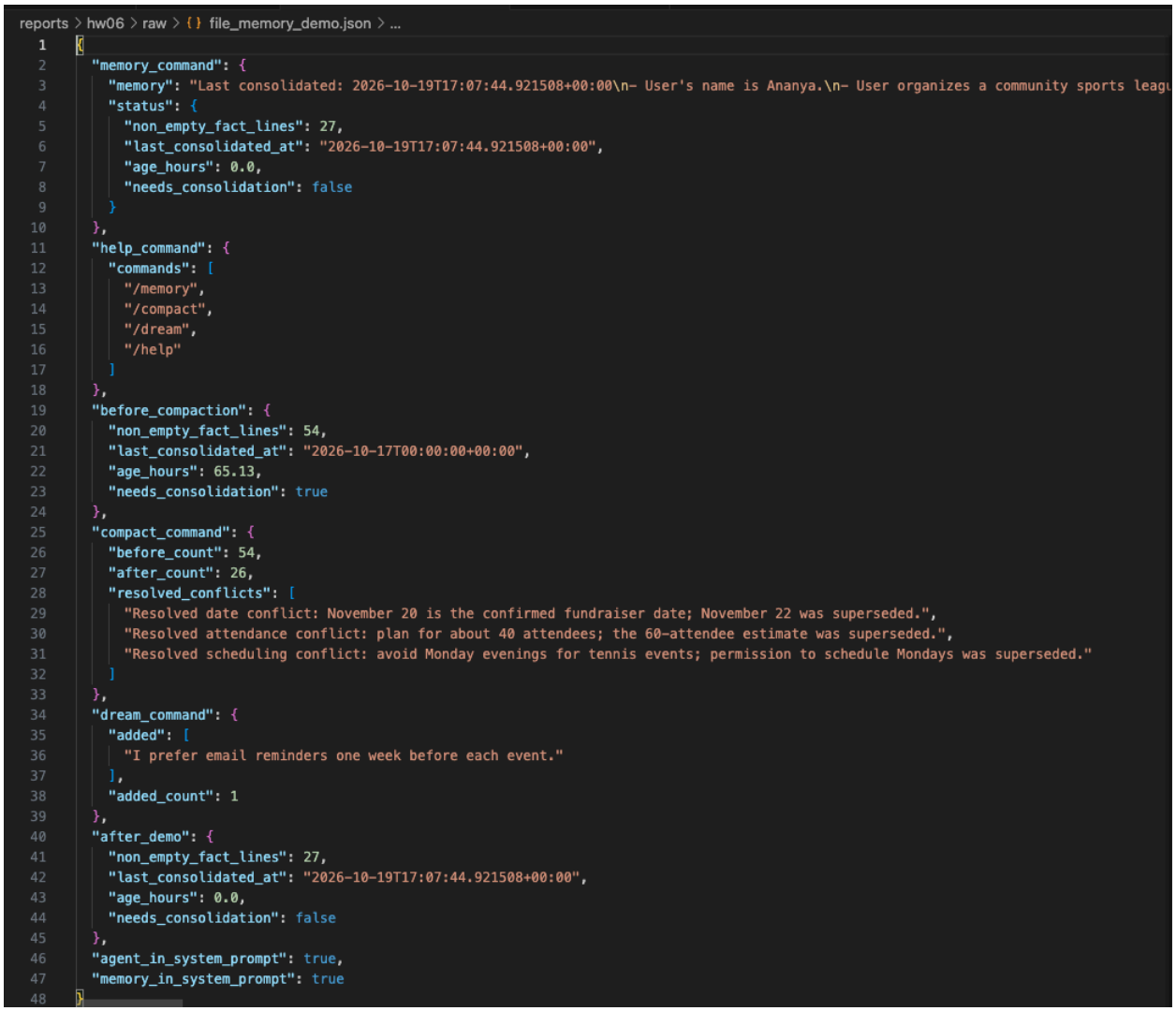
```python
def compact_memory() -> dict:
    before = len(fact_lines())
    retained = []
    seen = set()
    resolved = set()
    for line in fact_lines():
        fact = line.removeprefix("- ").strip()
        group = conflict_group(fact)
        if group:
            if group not in resolved:
                retained.append(CONFLICT_GROUPS[group]["resolution"])
                resolved.add(group)
            continue
        normalized = fact.casefold()
        if normalized not in seen:
            retained.append(fact)
            seen.add(normalized)

    timestamp = datetime.now(timezone.utc).isoformat()
    content = [f"{LAST_CONSOLIDATED}{timestamp}"] + [f"- {fact}" for fact in retained]
    MEMORY_PATH.write_text("\n".join(content) + "\n", encoding="utf-8")
    return {
        "before_count": before,
        "after_count": len(retained),
        "resolved_conflicts": [CONFLICT_GROUPS[group]["resolution"] for group in resolved],
    }
```

```python
def run_command(command: str, recent_messages: list[str] | None = None) -> dict:
    if command == "/memory":
        return {"memory": read_markdown(MEMORY_PATH), "status": consolidation_status()}
    if command == "/compact":
        return compact_memory()
    if command == "/dream":
        return dream(recent_messages or [])
    if command == "/help":
        return {"commands": ["/memory", "/compact", "/dream", "/help"]}
    raise ValueError("Unsupported memory command")
```

Ananya Yallapragada
019142383

file_memory_demo.json (output):



routers/file_memory.py

```python
@router.post("/command")
def file_memory_command(request: FileMemoryCommandRequest):
    try:
        return run_command(request.command, request.recent_messages)
    except ValueError as error:
        raise HTTPException(status_code=400, detail=str(error)) from error
```

Ananya Yallapragada
019142383

Terminal Output:



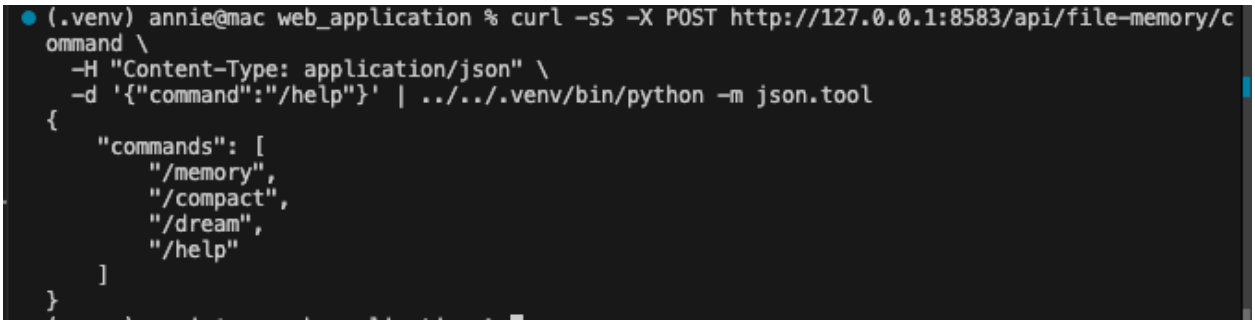
```
(.venv) annie@mac data260-2383 % cd code/web_application

../../.venv/bin/python run_file_memory_demo.py > /dev/null

../../.venv/bin/python -c '
import json
data = json.load(open("../../reports/hw06/raw/file_memory_demo.json"))
short = {
    "before_compaction": data["before_compaction"],
    "compact_command": data["compact_command"],
    "dream_command": data["dream_command"],
    "after_demo": data["after_demo"],
    "help_command": data["help_command"],
    "agent_in_system_prompt": data["agent_in_system_prompt"],
    "memory_in_system_prompt": data["memory_in_system_prompt"],
}
print(json.dumps(short, indent=2))
'

{
  "before_compaction": {
    "non_empty_fact_lines": 54,
    "last_consolidated_at": "2026-10-17T00:00:00+00:00",
    "age_hours": 65.13,
    "needs_consolidation": true
  },
  "compact_command": {
    "before_count": 54,
    "after_count": 26,
    "resolved_conflicts": [
      "Resolved date conflict: November 20 is the confirmed fundraiser date; November 22 was
 superseded.",
      "Resolved attendance conflict: plan for about 40 attendees; the 60-attendee estimate w
as superseded.",
      "Resolved scheduling conflict: avoid Monday evenings for tennis events; permission to
schedule Mondays was superseded."
    ]
  },
  "dream_command": {
    "added": [
      "I prefer email reminders one week before each event."
    ],
    "added_count": 1
  },
  "after_demo": {
    "non_empty_fact_lines": 27,
    "last_consolidated_at": "2026-10-19T17:07:44.921508+00:00",
    "age_hours": 0.0,
    "needs_consolidation": false
  },
  "help_command": {
    "commands": [
      "/memory",
      "/compact",
      "/dream",
      "/help"
    ]
  },
  "agent_in_system_prompt": true,
  "memory_in_system_prompt": true
}
```

/help endpoint



```
(.venv) annie@mac web_application % curl -sS -X POST http://127.0.0.1:8583/api/file-memory/c
ommand \
  -H "Content-Type: application/json" \
  -d '{"command":"/help"}' | ../../.venv/bin/python -m json.tool
{
    "commands": [
        "/memory",
        "/compact",
        "/dream",
        "/help"
    ]
}
```

Ananya Yallapragada
019142383

Criterion	MongoDB-backed memory	File-based memory
Inspectability	Queryable messages, summaries, and episodes; requires database tooling.	Directly readable in AGENT.md and MEMORY.md; simplest to inspect in Git.
Persistence after restart	Persists in MongoDB and is available after an API restart.	Persists as Markdown files and is available after an API restart.
Scalability across sessions	Supports indexed retrieval and many user/session records.	A single shared file becomes difficult to search, isolate, and merge at scale.
Facts that must not be forgotten	Better for user-scoped episodic facts and retrieval across sessions.	Better for a small set of explicitly curated, durable project rules and facts.

Consolidation Result

The controlled 55-line input had duplicate facts and three contradictions. '/compact' reduced 54 fact lines to 26 by removing duplicates and recording explicit resolutions. The final 'MEMORY.md' has 27 fact lines after '/dream' added one new durable preference, so it remains below the 40-line threshold. No contradiction was resolved incorrectly in this controlled test: the retained canonical values are November 20, about 40 attendees, and avoiding Monday evening tennis events.

Generative AI Use Disclosure

1. What I used an AI assistant for and what I did myself

I used an AI assistant to help interpret the homework requirements, plan the implementation, draft small code changes, and explain terminal commands. I personally ran the application, reviewed the raw outputs, captured the screenshots, updated the run log, and committed and pushed each incremental change. I also used it to polish the report for sentence and grammar issues.

2. One AI-produced output that was wrong/unsuitable, or one thing I independently verified

Ananya Yallapragada
019142383

During the MongoDB memory demonstration, the local model incorrectly changed the fundraiser plan by suggesting alternative non-Monday dates even though the user had stated November 20. It also sometimes applied a fundraiser detail, such as bringing water, to unrelated tennis events.

3. How I detected the problem or verified the result

I reviewed the saved conversation and memory JSON instead of relying only on a fluent response. The generated summary and reply text showed the unsupported date change and the inappropriate transfer of fundraiser details to tennis scheduling.

4. What I changed and why it works now

I strengthened the extraction, summary, and response prompts to preserve stated dates, times, venues, and preferences exactly, not infer missing details, and report conflicts rather than silently resolve them. I also added fixed probes and deterministic scoring. The final experiment replies retained the stated fundraiser date, time, and venue, and the saved probe scores make remaining omissions visible.